5

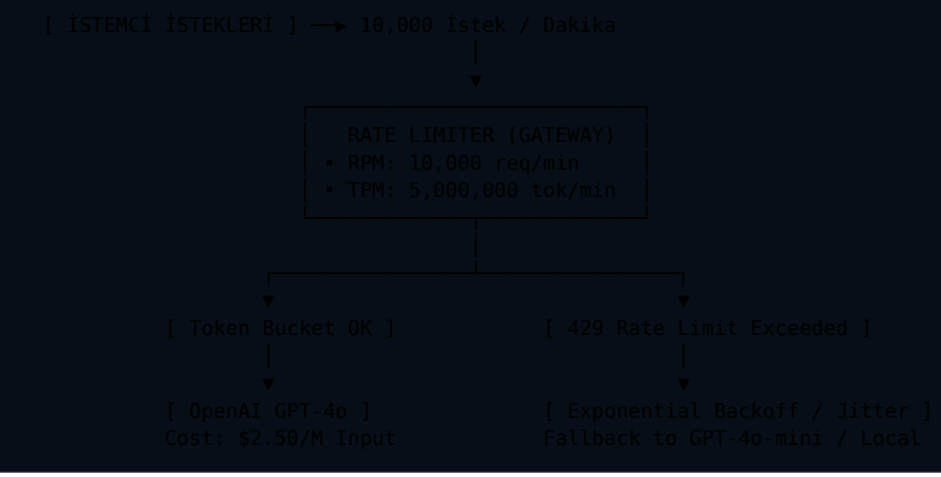
BÖLÜM: AI ENGINEER ROLÜ, LLM ANATOMISI & MAANG MÜLAKAT ÇERÇEVESI | MODULE: THE AI ENGINEER ROLE, LLM ANATOMY & THE MAANG INTERVIEW FRAMEWORK

LLM API Maliyet Analizi, Token Hesaplama & Rate Limiting (TPM / RPM)

LLM Unit Economics: Token Economics, Cost Modeling & Rate Limits (TPM / RPM)

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir şirkette herkese limitsiz kurumsal kredi kartı dağıttığınızı ve her çalışanın her kahve aldığında 100 dolar harcadığını düşünün; ay sonu şirket iflas eder! LLM API'leri de böyledir: Her soru sorduğunuzda ve her cevap aldığınızda OpenAI'a kelime kelime para ödersiniz. Eğer 100.000 kullanıcınız aynı anda 'Şu 50 sayfalık dosyayı özetle' derse faturanız bir gecede 10.000 doları bulabilir! Ayrıca OpenAI size 'Dakikada en fazla 5 Milyon Token harcayabilirsin' sınırı koyar (TPM). Bu sınırı aşarsanız sisteminiz hata verir!

🇬🇧 ENGLISH (ELI5)

Imagine giving every employee an unrestricted corporate credit card where every single coffee costs $50. Unmonitored, your company goes bankrupt in 30 days! LLM APIs charge you micro-pennies for every individual input and output token. If 100,000 concurrent users request 50-page document summaries simultaneously, your daily bill will exceed $10,000. Furthermore, OpenAI enforces Tokens Per Minute (TPM) and Requests Per Minute (RPM) quotas; breaching them results in devastating HTTP 429 cascade failures!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Seviyesinde Birim Maliyet ve Kapasite Yönetimi: 1) Input vs Output Maliyeti: LLM'lerde çıktı token'ları (Output), girdi token'larından (Input) 3 ila 4 kat daha pahalıdır (GPT-4o: $2.50 Input / $10.00 Output per 1M). Bu yüzden prompt'larda 'öz ve kısa yanıt üret' talimatı maliyeti %70 düşürür. 2) TPM / RPM Kotası: 1 Tier-5 kurumsal hesapta dahi 10M TPM limiti bulunabilir. Zirve yüklerde (Peak Traffic) Token Bucket algoritmasıyla istemciler sıraya alınmalı veya gpt-4o-mini'ye yönlendirilmelidir (Model Downgrading). 3) Prompt Caching: OpenAI 1024 token üzerindeki tekrarlayan prefix prompt'lara %50 indirim uygular. Sistem prompt'larını dinamik değil, sabit tutmak büyük tasarruf sağlar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Unit economics and throughput engineering at MAANG scale: 1) Asymmetric Pricing: Completion tokens cost 3x-4x more than prompt tokens due to serialized GPU execution. Prompt instructions enforcing conciseness directly cut completion costs. 2) Quota Throttling (TPM/RPM): Even Tier-5 OpenAI organizations face concurrency ceilings. Implement client-side token bucket rate limiters to buffer bursts. 3) Prompt Caching: OpenAI automatically discounts identical prompt prefixes >= 1,024 tokens by 50-80%, demanding deterministic, cached static system prompts across user sessions.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25: Token ve Bütçe Koruma Bekçisi (Token Budget Guard)
import java.util.concurrent.atomic.AtomicLong;

public class TokenBudgetGovernor {
    private final long dailyBudgetTokens;
    private final AtomicLong consumedTokens = new AtomicLong(0);

    public TokenBudgetGovernor(long dailyBudgetTokens) {
        this.dailyBudgetTokens = dailyBudgetTokens;
    }

    public synchronized boolean authorizeRequest(int estimatedTokens) {
        if (consumedTokens.get() + estimatedTokens > dailyBudgetTokens) {
            return false; // Bütçe aşıldı! İstek ucuz modele veya önbelleğe yönlendirilmeli
        }
        consumedTokens.addAndGet(estimatedTokens);
        return true;
    }

    public long getRemainingBudget() {
        return Math.max(0, dailyBudgetTokens - consumedTokens.get());
    }
}

void main() {
    var governor = new TokenBudgetGovernor(1_000_000); // Günlük 1M Token Sınırı
    int requestTokens = 25_000;

    IO.println("İstek İzni: " + (governor.authorizeRequest(requestTokens) ? "✅ ONAYLANDI" : "❌ BÜTÇE
AŞILDI"));
    IO.println("Kalan Token Bütçesi: " + governor.getRemainingBudget());
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

En büyük mimari hata: Streaming kullanırken istemci bağlantıyı kesse bile arka planda modelin token üretmeye ve fatura yazmaya devam etmesidir. İstemci koptuğunda (Client Disconnect) `Flux.cancel()` ile LLM çağrısı anında iptal edilmelidir!

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Production hazard: If a user closes their browser tab mid-stream, unmanaged backend threads continue consuming LLM tokens. Reactive cancellation signals (`Flux.doOnCancel`) must immediately abort the upstream OpenAI connection.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Output token'ları her zaman input token'larından 3-4 kat daha pahalıdır.
- Sabit sistem prompt'ları OpenAI Prompt Caching sayesinde maliyeti yarı yarıya düşürür.
- Üretim ortamında daima Token/Request bazlı Circuit Breaker ve Fallback mekanizmaları kurulmalıdır.

🎯 Key Takeaways for the Interview

- Output tokens are significantly more expensive than input tokens due to sequential GPU decoding.
- Static system prompts leverage OpenAI Prompt Caching to slash latency and costs by up to 50%.
- Production architectures require Token-level rate limiters and graceful fallback policies.